

<- This means this lab is on your local computer

Lab 4: Calling Convention

CS61C SU26

Topics

- Why Calling Convention
- Caller- v.s. Callee-saved registers
- How to save registers onto the stack
- t, s, a registers
- Calling Convention RISC-V sample code walkthrough

RISC-V Calling Convention and the Stack

- For your code to be portable, a calling convention on how the registers are used and saved must be set.
- RISC-V only has 32 registers, you'll run out of registers when writing complex programs
- You'd need to save register values on the stack.
- RISC-V calling convention: who is responsible in saving the registers into the stack?
 - Calling convention can be boiled down to the rules used for communication between functions

#	Name	Description	#	Name	Desc
x0	zero	Constant 0	x16	a6	Args
x1	ra	Return Address	x17	a7	
x2	sp	Stack Pointer	x18	s2	Saved Registers
x3	gp	Global Pointer	x19	s3	
x4	tp	Thread Pointer	x20	s4	
x5	t0	Temporary Registers	x21	s5	
x6	t1		x22	s6	
x7	t2		x23	s7	
x8	s0	Saved Registers	x24	s8	
x9	s1		x25	s9	
x10	a0	Function Arguments or Return Values	x26	s10	
x11	a1		x27	s11	
x12	a2	Function Arguments	x28	t3	Temporaries
x13	a3		x29	t4	
x14	a4		x30	t5	
x15	a5		x31	t6	
Caller saved registers					
Callee saved registers (except x0, gp, tp)					

Caller vs Callee

<i>Caller saved registers</i>
Callee saved registers (except x0 , gp , tp)

- Caller
 - A function that calls another function
- Callee
 - A function that gets called by another function
- A function can be both a caller and a callee at the same time.

Distinction: Caller vs Callee Saved Registers

- **Caller Saved**
 - **caller** does not expect the register to hold the same value after function call, so it saves the value stored in the register preemptively
 - considered to be changed by the func callee when returned to the caller
 - e.g. argument registers (**a0-a7**), temporary registers (**t0-t6**), **ra**
- **Callee Saved:**
 - **caller** expects for the register to hold the same values after the function calls or the **callee** executes
 - e.g. saved registers (**s0-s11**), stack pointer (**sp**)
 - Saved in prologue, loaded in epilogue

How to save registers onto the stack

```
addi sp sp -4 # push the stack one word (4 bytes) down
sw ra 0(sp)    # save ra at the 0th offset
jal ra func    # call some function, ra is changed now
lw ra 0(sp)    # load ra from the 0th offset of the stack
addi sp sp 4   # push the stack pointer back to where it was
jr ra         # return from current function
```

How to save registers onto the stack

```
addi sp sp -4 # push the stack one word (4 bytes) down
sw ra 0(sp)   # save ra at the 0th offset
jal ra func   # call some function, ra is changed now
lw ra 0(sp)   # load ra from the 0th offset of the stack
addi sp sp 4   # push the stack pointer back to where it was
jr ra         # return from current function
```

ra	0x61C0
----	--------



How to save registers onto the stack

```
addi sp sp -4 # push the stack one word (4 bytes) down
sw ra 0(sp)   # save ra at the 0th offset
jal ra func   # call some function, ra is changed now
lw ra 0(sp)   # load ra from the 0th offset of the stack
addi sp sp 4  # push the stack pointer back to where it was
jr ra        # return from current function
```

ra	0x61C0
----	--------

Stack

sp →



How to save registers onto the stack

```
addi sp sp -4 # push the stack one word (4 bytes) down
```

```
sw ra 0(sp) # save ra at the 0th offset
```

```
jal ra func # call some function, ra is changed now
```

```
lw ra 0(sp) # load ra from the 0th offset of the stack
```

```
addi sp sp 4 # push the stack pointer back to where it was
```

```
jr ra # return from current function
```

Stack

sp →



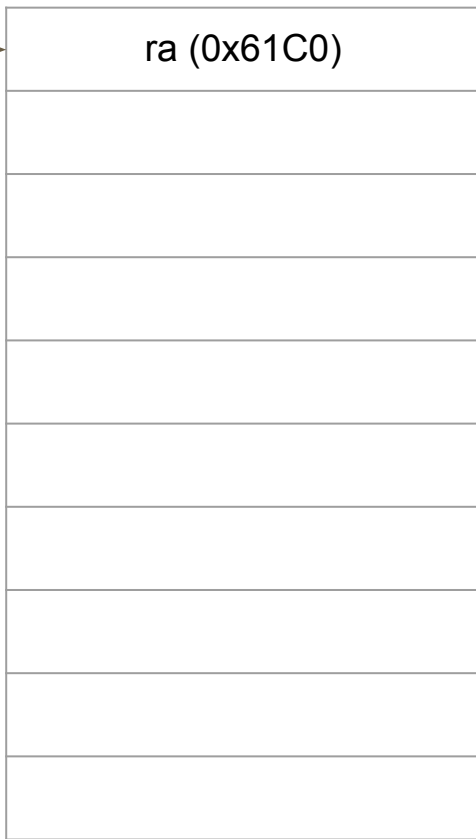
ra	0x61C0
----	--------

How to save registers onto the stack

```
addi sp sp -4 # push the stack one word (4 bytes) down
sw ra 0(sp)   # save ra at the 0th offset
jal ra func   # call some function, ra is changed now
lw ra 0(sp)   # load ra from the 0th offset of the stack
addi sp sp 4   # push the stack pointer back to where it was
jr ra         # return from current function
```

sp →

Stack



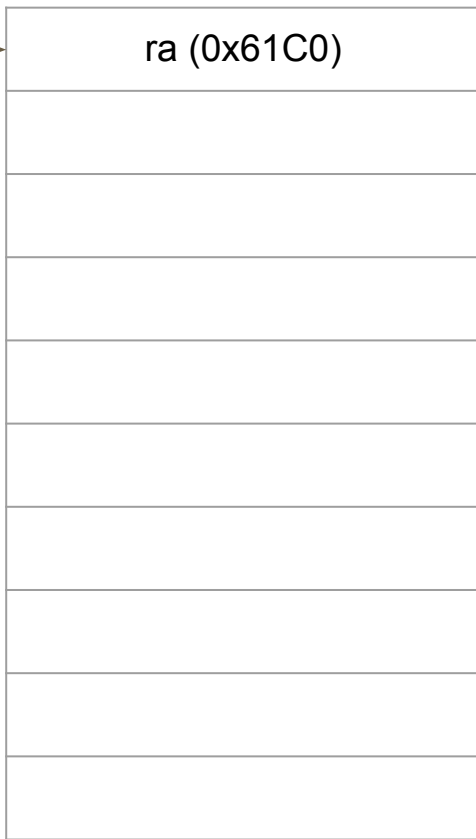
ra	???
----	-----

How to save registers onto the stack

```
addi sp sp -4 # push the stack one word (4 bytes) down
sw ra 0(sp)   # save ra at the 0th offset
jal ra func   # call some function, ra is changed now
lw ra 0(sp)   # load ra from the 0th offset of the stack
addi sp sp 4  # push the stack pointer back to where it was
jr ra        # return from current function
```

sp →

Stack



ra	0x61C0
----	--------

How to save registers onto the stack

```
addi sp sp -4 # push the stack one word (4 bytes) down
sw ra 0(sp)   # save ra at the 0th offset
jal ra func   # call some function, ra is changed now
lw ra 0(sp)   # load ra from the 0th offset of the stack
addi sp sp 4   # push the stack pointer back to where it was
jr ra        # return from current function
```

sp →

Stack

ra (0x61C0)

ra	0x61C0
----	--------

How to save registers onto the stack

- The element on the stack doesn't get removed, but sp gets moved in a way that makes ra inaccessible *if you follow calling convention*

sp →

Stack

ra (0x61C0)

ra	0x61C0
----	--------

t-registers

- “Temporary” registers
- Caller saved
 - Mess with them all you like
 - Assume their values will be totally different after you call a function

s-registers

- “Saved” registers
- Callee saved
 - Need to be saved and restored during a function call
 - Caller expects these values to be the same after the func call -> callee can alter the value saved IF it restores it in the function call

a-registers

- “Argument” (but also return) registers
- Caller Saved
 - Technically you can mess with them all you like but you should generally just use them for arguments to functions and return values of your function
 - If you call a function you should assume the values will be different but you can also assume that any return values will be stored in these functions

Example

```
1  main: # uses t0, s0
2      # do something here
3      addi sp, sp, -4
4      sw t0, 0(sp)
5      jal ra, func1
6      lw t0, 0(sp)
7      addi sp, sp, 4
8      # do something here
9      j exit
10 func1: # uses t0, s0
11     addi sp, sp, -8
12     sw s0, 0(sp)
13     sw ra, 4(sp)
14     # do something here
15     addi sp, sp, -4
16     sw t0, 0(sp)
17     jal ra, func2
18     lw t0, 0(sp)
19     addi sp, sp, 4
20     # do something here
21     lw ra, 4(sp)
22     lw s0, 0(sp)
23     addi sp, sp, 8
24     jr ra
25 func2: # uses t0, s0
26     addi sp, sp, -4
27     sw s0, 0(sp)
28     # do something here
29     lw s0, 0(sp)
30     addi sp, sp, 4
31     jr ra
32 exit:
```

Example

#	Name	Description	#	Name	Desc
x0	zero	Constant 0	x16	a6	Args
x1	ra	Return Address	x17	a7	
x2	sp	Stack Pointer	x18	s2	Saved Registers
x3	gp	Global Pointer	x19	s3	
x4	tp	Thread Pointer	x20	s4	
x5	t0	Temporary Registers	x21	s5	
x6	t1		x22	s6	
x7	t2		x23	s7	
x8	s0	Saved Registers	x24	s8	
x9	s1		x25	s9	
x10	a0	Function Arguments or Return Values	x26	s10	
x11	a1		x27	s11	
x12	a2	Function Arguments	x28	t3	Temporaries
x13	a3		x29	t4	
x14	a4		x30	t5	
x15	a5		x31	t6	
Caller saved registers					
Callee saved registers (except x0, gp, tp)					

```

1  main: # uses t0, s0
2      # do something here
3      addi sp, sp, -4
4      sw t0, 0(sp)
5      jal ra, func1
6      lw t0, 0(sp)
7      addi sp, sp, 4
8      # do something here
9      j exit
10 func1: # uses t0, s0
11     addi sp, sp, -8
12     sw s0, 0(sp)
13     sw ra, 4(sp)
14     # do something here
15     addi sp, sp, -4
16     sw t0, 0(sp)
17     jal ra, func2
18     lw t0, 0(sp)
19     addi sp, sp, 4
20     # do something here
21     lw ra, 4(sp)
22     lw s0, 0(sp)
23     addi sp, sp, 8
24     jr ra
25 func2: # uses t0, s0
26     addi sp, sp, -4
27     sw s0, 0(sp)
28     # do something here
29     lw s0, 0(sp)
30     addi sp, sp, 4
31     jr ra
32 exit:

```

- main will call func1
- As the caller, it's responsible in saving t0
- func1 is called by main
- As the callee, it's responsible in saving s0
- func1 will call func2
- As the caller, it's responsible in saving ra and t0 as well
- func2 is called by func1
- As the callee, it's responsible in saving s0

Example

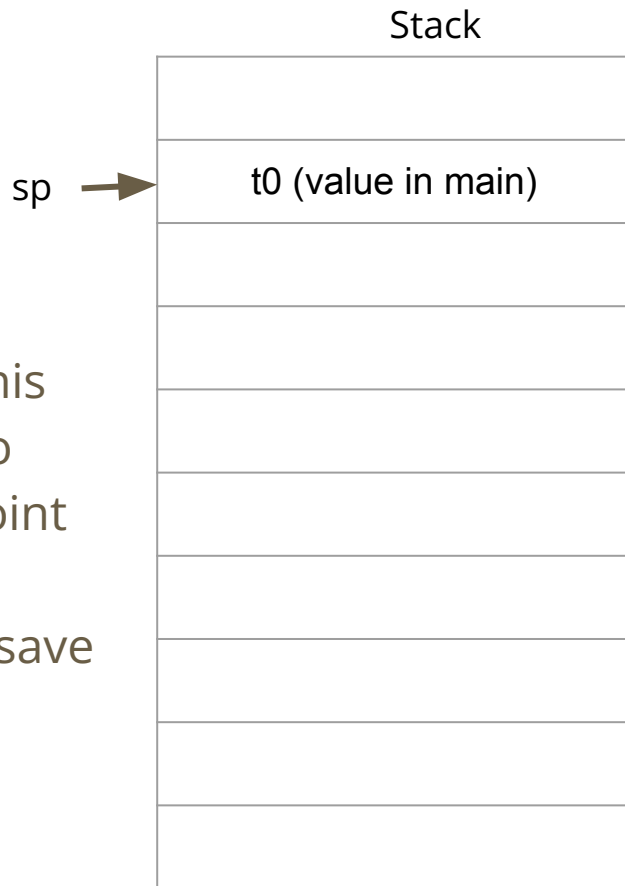
#	Name	Description	#	Name	Desc
x0	zero	Constant 0	x16	a6	Args
x1	ra	Return Address	x17	a7	
x2	sp	Stack Pointer	x18	s2	Saved Registers
x3	gp	Global Pointer	x19	s3	
x4	tp	Thread Pointer	x20	s4	
x5	t0	Temporary Registers	x21	s5	
x6	t1		x22	s6	
x7	t2		x23	s7	
x8	s0	Saved Registers	x24	s8	
x9	s1		x25	s9	
x10	a0	Function Arguments or Return Values	x26	s10	
x11	a1		x27	s11	
x12	a2	Function Arguments	x28	t3	Temporaries
x13	a3		x29	t4	
x14	a4		x30	t5	
x15	a5		x31	t6	
Caller saved registers					
Callee saved registers (except x0, gp, tp)					

```

1  main: # uses t0, s0
2      # do something here
3      addi sp, sp, -4
4      sw t0, 0(sp)
5      jal ra, func1
6      lw t0, 0(sp)
7      addi sp, sp, 4
8      # do something here
9      j exit
10 func1: # uses t0, s0
11     addi sp, sp, -8
12     sw s0, 0(sp)
13     sw ra, 4(sp)
14     # do something here
15     addi sp, sp, -4
16     sw t0, 0(sp)
17     jal ra, func2
18     lw t0, 0(sp)
19     addi sp, sp, 4
20     # do something here
21     lw ra, 4(sp)
22     lw s0, 0(sp)
23     addi sp, sp, 8
24     jr ra
25 func2: # uses t0, s0
26     addi sp, sp, -4
27     sw s0, 0(sp)
28     # do something here
29     lw s0, 0(sp)
30     addi sp, sp, 4
31     jr ra
32 exit:

```

- ra will be modified at this point: jump to func1 then point ra to main
- As the caller, save t0



Example

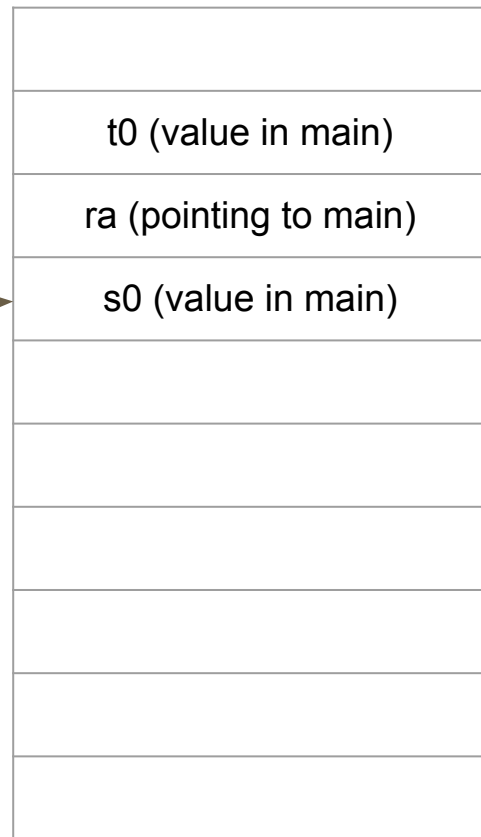
#	Name	Description	#	Name	Desc
x0	zero	Constant 0	x16	a6	Args
x1	ra	Return Address	x17	a7	
x2	sp	Stack Pointer	x18	s2	Saved Registers
x3	gp	Global Pointer	x19	s3	
x4	tp	Thread Pointer	x20	s4	
x5	t0	Temporary Registers	x21	s5	
x6	t1		x22	s6	
x7	t2		x23	s7	
x8	s0	Saved Registers	x24	s8	
x9	s1		x25	s9	
x10	a0	Function Arguments or Return Values	x26	s10	
x11	a1		x27	s11	
x12	a2	Function Arguments	x28	t3	Temporaries
x13	a3		x29	t4	
x14	a4		x30	t5	
x15	a5		x31	t6	
Caller saved registers					
Callee saved registers (except x0, gp, tp)					

```

1  main: # uses t0, s0
2      # do something here
3      addi sp, sp, -4
4      sw t0, 0(sp)
5      jal ra, func1
6      lw t0, 0(sp)
7      addi sp, sp, 4
8      # do something here
9      j exit
10 func1: # uses t0, s0
11     addi sp, sp, -8
12     sw s0, 0(sp)
13     sw ra, 4(sp)
14     # do something here
15     addi sp, sp, -4
16     sw t0, 0(sp)
17     jal ra, func2
18     lw t0, 0(sp)
19     addi sp, sp, 4
20     # do something here
21     lw ra, 4(sp)
22     lw s0, 0(sp)
23     addi sp, sp, 8
24     jr ra
25 func2: # uses t0, s0
26     addi sp, sp, -4
27     sw s0, 0(sp)
28     # do something here
29     lw s0, 0(sp)
30     addi sp, sp, 4
31     jr ra
32 exit:

```

Stack



- s0 and ra can be safely modified by func1 after this point
- As callee, can modify t0

Example

#	Name	Description	#	Name	Desc
x0	zero	Constant 0	x16	a6	Args
x1	ra	Return Address	x17	a7	
x2	sp	Stack Pointer	x18	s2	Saved Registers
x3	gp	Global Pointer	x19	s3	
x4	tp	Thread Pointer	x20	s4	
x5	t0	Temporary Registers	x21	s5	
x6	t1		x22	s6	
x7	t2		x23	s7	
x8	s0	Saved Registers	x24	s8	
x9	s1		x25	s9	
x10	a0	Function Arguments or Return Values	x26	s10	
x11	a1		x27	s11	
x12	a2	Function Arguments	x28	t3	Temporaries
x13	a3		x29	t4	
x14	a4		x30	t5	
x15	a5		x31	t6	
Caller saved registers					
Callee saved registers (except x0, gp, tp)					

```

1  main: # uses t0, s0
2      # do something here
3      addi sp, sp, -4
4      sw t0, 0(sp)
5      jal ra, func1
6      lw t0, 0(sp)
7      addi sp, sp, 4
8      # do something here
9      j exit
10 func1: # uses t0, s0
11     addi sp, sp, -8
12     sw s0, 0(sp)
13     sw ra, 4(sp)
14     # do something here
15     addi sp, sp, -4
16     sw t0, 0(sp)
17     jal ra, func2
18     lw t0, 0(sp)
19     addi sp, sp, 4
20     # do something here
21     lw ra, 4(sp)
22     lw s0, 0(sp)
23     addi sp, sp, 8
24     jr ra
25 func2: # uses t0, s0
26     addi sp, sp, -4
27     sw s0, 0(sp)
28     # do something here
29     lw s0, 0(sp)
30     addi sp, sp, 4
31     jr ra
32 exit:

```

- ra will be modified at this point: jump to func2 then point ra to func1
- As the caller, save t0

sp →

Stack

t0 (value in main)
ra (pointing to main)
s0 (value in main)
t0 (value in func1)

Example

#	Name	Description	#	Name	Desc
x0	zero	Constant 0	x16	a6	Args
x1	ra	Return Address	x17	a7	
x2	sp	Stack Pointer	x18	s2	Saved Registers
x3	gp	Global Pointer	x19	s3	
x4	tp	Thread Pointer	x20	s4	
x5	t0	Temporary Registers	x21	s5	
x6	t1		x22	s6	
x7	t2		x23	s7	
x8	s0	Saved Registers	x24	s8	
x9	s1		x25	s9	
x10	a0	Function Arguments or Return Values	x26	s10	
x11	a1		x27	s11	
x12	a2	Function Arguments	x28	t3	Temporaries
x13	a3		x29	t4	
x14	a4		x30	t5	
x15	a5		x31	t6	
Caller saved registers					
Callee saved registers (except x0, gp, tp)					

```

1  main: # uses t0, s0
2      # do something here
3      addi sp, sp, -4
4      sw t0, 0(sp)
5      jal ra, func1
6      lw t0, 0(sp)
7      addi sp, sp, 4
8      # do something here
9      j exit
10 func1: # uses t0, s0
11     addi sp, sp, -8
12     sw s0, 0(sp)
13     sw ra, 4(sp)
14     # do something here
15     addi sp, sp, -4
16     sw t0, 0(sp)
17     jal ra, func2
18     lw t0, 0(sp)
19     addi sp, sp, 4
20     # do something here
21     lw ra, 4(sp)
22     lw s0, 0(sp)
23     addi sp, sp, 8
24     jr ra
25 func2: # uses t0, s0
26     addi sp, sp, -4
27     sw s0, 0(sp)
28     # do something here
29     lw s0, 0(sp)
30     addi sp, sp, 4
31     jr ra
32 exit:

```

sp →

- s0 can be safely modified by func2 after this point
- As callee, can modify t0

Stack

t0 (value in main)
ra (pointing to main)
s0 (value in main)
t0 (value in func1)
s0 (value in func1)

Example

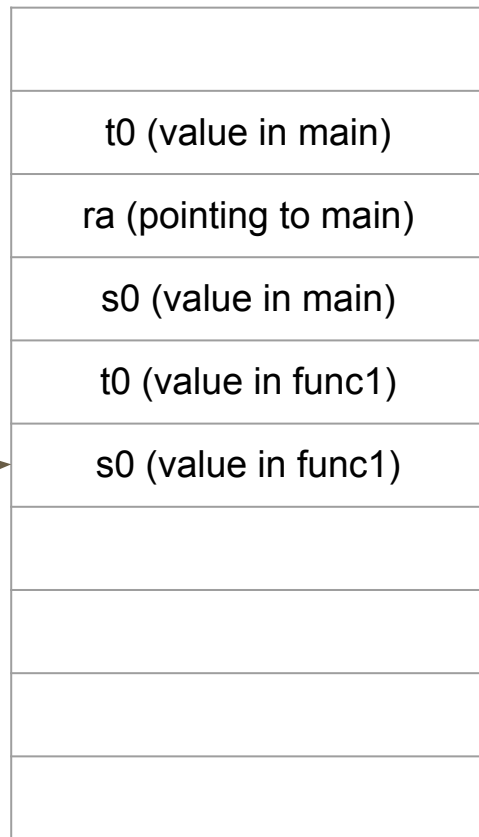
#	Name	Description	#	Name	Desc
x0	zero	Constant 0	x16	a6	Args
x1	ra	Return Address	x17	a7	
x2	sp	Stack Pointer	x18	s2	Saved Registers
x3	gp	Global Pointer	x19	s3	
x4	tp	Thread Pointer	x20	s4	
x5	t0	Temporary Registers	x21	s5	
x6	t1		x22	s6	
x7	t2		x23	s7	
x8	s0	Saved Registers	x24	s8	
x9	s1		x25	s9	
x10	a0	Function Arguments or Return Values	x26	s10	
x11	a1		x27	s11	
x12	a2	Function Arguments	x28	t3	Temporaries
x13	a3		x29	t4	
x14	a4		x30	t5	
x15	a5		x31	t6	
Caller saved registers					
Callee saved registers (except x0, gp, tp)					

```

1  main: # uses t0, s0
2      # do something here
3      addi sp, sp, -4
4      sw t0, 0(sp)
5      jal ra, func1
6      lw t0, 0(sp)
7      addi sp, sp, 4
8      # do something here
9      j exit
10 func1: # uses t0, s0
11     addi sp, sp, -8
12     sw s0, 0(sp)
13     sw ra, 4(sp)
14     # do something here
15     addi sp, sp, -4
16     sw t0, 0(sp)
17     jal ra, func2
18     lw t0, 0(sp)
19     addi sp, sp, 4
20     # do something here
21     lw ra, 4(sp)
22     lw s0, 0(sp)
23     addi sp, sp, 8
24     jr ra
25 func2: # uses t0, s0
26     addi sp, sp, -4
27     sw s0, 0(sp)
28     # do something here
29     lw s0, 0(sp)
30     addi sp, sp, 4
31     jr ra
32 exit:

```

Stack



sp →

- load saved s0
- ra points to func1, go back

Example

#	Name	Description	#	Name	Desc
x0	zero	Constant 0	x16	a6	Args
x1	ra	Return Address	x17	a7	
x2	sp	Stack Pointer	x18	s2	Saved Registers
x3	gp	Global Pointer	x19	s3	
x4	tp	Thread Pointer	x20	s4	
x5	t0	Temporary Registers	x21	s5	
x6	t1		x22	s6	
x7	t2		x23	s7	
x8	s0	Saved Registers	x24	s8	
x9	s1		x25	s9	
x10	a0	Function Arguments or Return Values	x26	s10	
x11	a1		x27	s11	
x12	a2	Function Arguments	x28	t3	
x13	a3		x29	t4	
x14	a4		x30	t5	
x15	a5		x31	t6	
Caller saved registers					
Callee saved registers (except x0, gp, tp)					

```

1  main: # uses t0, s0
2      # do something here
3      addi sp, sp, -4
4      sw t0, 0(sp)
5      jal ra, func1
6      lw t0, 0(sp)
7      addi sp, sp, 4
8      # do something here
9      j exit
10 func1: # uses t0, s0
11     addi sp, sp, -8
12     sw s0, 0(sp)
13     sw ra, 4(sp)
14     # do something here
15     addi sp, sp, -4
16     sw t0, 0(sp)
17     jal ra, func2
18     lw t0, 0(sp)
19     addi sp, sp, 4
20     # do something here
21     lw ra, 4(sp)
22     lw s0, 0(sp)
23     addi sp, sp, 8
24     jr ra
25 func2: # uses t0, s0
26     addi sp, sp, -4
27     sw s0, 0(sp)
28     # do something here
29     lw s0, 0(sp)
30     addi sp, sp, 4
31     jr ra
32 exit:

```

sp →

- s0 will look unmodified
- load saved t0
- t0 will look unmodified

Stack

t0 (value in main)
ra (pointing to main)
s0 (value in main)
t0 (value in func1)
s0 (value in func1)

Example

#	Name	Description	#	Name	Desc
x0	zero	Constant 0	x16	a6	Args
x1	ra	Return Address	x17	a7	
x2	sp	Stack Pointer	x18	s2	Saved Registers
x3	gp	Global Pointer	x19	s3	
x4	tp	Thread Pointer	x20	s4	
x5	t0	Temporary Registers	x21	s5	
x6	t1		x22	s6	
x7	t2		x23	s7	
x8	s0	Saved Registers	x24	s8	
x9	s1		x25	s9	
x10	a0	Function Arguments or Return Values	x26	s10	
x11	a1		x27	s11	
x12	a2	Function Arguments	x28	t3	Temporaries
x13	a3		x29	t4	
x14	a4		x30	t5	
x15	a5		x31	t6	
Caller saved registers					
Callee saved registers (except x0, gp, tp)					

```

1  main: # uses t0, s0
2      # do something here
3      addi sp, sp, -4
4      sw t0, 0(sp)
5      jal ra, func1
6      lw t0, 0(sp)
7      addi sp, sp, 4
8      # do something here
9      j exit
10 func1: # uses t0, s0
11     addi sp, sp, -8
12     sw s0, 0(sp)
13     sw ra, 4(sp)
14     # do something here
15     addi sp, sp, -4
16     sw t0, 0(sp)
17     jal ra, func2
18     lw t0, 0(sp)
19     addi sp, sp, 4
20     # do something here
21     lw ra, 4(sp)
22     lw s0, 0(sp)
23     addi sp, sp, 8
24     jr ra
25 func2: # uses t0, s0
26     addi sp, sp, -4
27     sw s0, 0(sp)
28     # do something here
29     lw s0, 0(sp)
30     addi sp, sp, 4
31     jr ra
32 exit:

```

sp →

- load saved ra, can now go back to main
- load saved s0

Stack

t0 (value in main)
ra (pointing to main)
s0 (value in main)
t0 (value in func1)
s0 (value in func1)

Example

#	Name	Description	#	Name	Desc
x0	zero	Constant 0	x16	a6	Args
x1	ra	Return Address	x17	a7	
x2	sp	Stack Pointer	x18	s2	Saved Registers
x3	gp	Global Pointer	x19	s3	
x4	tp	Thread Pointer	x20	s4	
x5	t0	Temporary Registers	x21	s5	
x6	t1		x22	s6	
x7	t2		x23	s7	
x8	s0	Saved Registers	x24	s8	
x9	s1		x25	s9	
x10	a0	Function Arguments or Return Values	x26	s10	
x11	a1		x27	s11	
x12	a2	Function Arguments	x28	t3	
x13	a3		x29	t4	
x14	a4		x30	t5	
x15	a5		x31	t6	
Caller saved registers					
Callee saved registers (except x0, gp, tp)					

```

1  main: # uses t0, s0
2      # do something here
3      addi sp, sp, -4
4      sw t0, 0(sp)
5      jal ra, func1
6      lw t0, 0(sp)
7      addi sp, sp, 4
8      # do something here
9      j exit
10 func1: # uses t0, s0
11     addi sp, sp, -8
12     sw s0, 0(sp)
13     sw ra, 4(sp)
14     # do something here
15     addi sp, sp, -4
16     sw t0, 0(sp)
17     jal ra, func2
18     lw t0, 0(sp)
19     addi sp, sp, 4
20     # do something here
21     lw ra, 4(sp)
22     lw s0, 0(sp)
23     addi sp, sp, 8
24     jr ra
25 func2: # uses t0, s0
26     addi sp, sp, -4
27     sw s0, 0(sp)
28     # do something here
29     lw s0, 0(sp)
30     addi sp, sp, 4
31     jr ra
32 exit:

```

- s0 will look unmodified
- load saved t0
- t0 will look unmodified
- exit program

